

TP N°1

Compétences acquises dans ce TP

L'éditeur de texte nano.

Création de scripts python 3.

Modalités

Durée 4 heures, seul ou en binôme, non noté.

Ordinateur personnel obligatoire.

Pour des raisons de sécurité informatique, ce TP ne peut être effectué que si votre ordinateur est connecté sur le réseau WiFi appelé « JUNIA_STUDENTS ».

Introduction

Dans ce TP vous allez vous connecter au serveur Linux d'adresse IP 10.34.164.21. Utilisez l'identifiant présent dans le fichier Excel présent sur Junia-learning.

Vous allez utiliser le shell Bash pour lancer un l'éditeur de texte nano. Cet éditeur vous servira pour créer vos scripts python.

L'éditeur de texte nano

Un éditeur de texte est un programme qui permet de modifier des fichiers de texte brut, sans mise en forme (gras, italique, souligné...). Sous Windows, on dispose d'un éditeur de texte très basique : le Bloc-Notes. Sous Linux, on a le choix entre Nano, Vim, Emacs et bien d'autres, sachant qu'au moins un de ceux-là est installé par défaut sur la plupart des distributions.

Les éditeurs de texte servent... à éditer du texte ! Par exemple des fichiers de configuration, code sources de programmes, etc.

Utilisation de l'éditeur de texte nano

Comment installer Nano ?

L'éditeur nano est disponible par défaut dans toutes les distributions Linux les plus populaires et vous pouvez l'exécuter avec une simple commande :

```
nano
```

La commande ci-dessus ouvrira simplement un nouveau fichier. Vous pouvez taper dans la fenêtre, enregistrer le fichier et quitter.

Comment ouvrir un nouveau fichier et lui donner un nom en utilisant Nano ?

Bien que le simple fait d'exécuter nano soit acceptable, vous pouvez donner un nom à votre document avant de commencer. Pour ce faire, nommez simplement le nom du fichier après la commande nano.

La commande principale que nous allons utiliser est alors:

```
nano nomfichier
```

Après avoir exécuté la commande, vous serez dirigé vers la fenêtre de l'éditeur, où vous pourrez maintenant utiliser et éditer votre texte avec l'éditeur de texte **nano**. Utilisez les touches fléchées de votre clavier pour déplacer le curseur.

Au bas de cette fenêtre, vous verrez certains des raccourcis utilisables avec l'éditeur **nano**. Le symbole ^ signifie que vous devez appuyer sur **CTRL + [Touche]** pour lancer la commande choisie.

Vous trouverez ci-dessous quelques-unes des principales commandes. Vous pouvez consulter la liste complète sur internet.

Commande	Explication
CTRL + A	Aller au début de la ligne
CTRL + E	Aller à la fin de la ligne.
CTRL + Y	Faire défiler la page vers le haut.
CTRL + V	Faire défiler la page vers le bas.
CTRL + O	Pour sauvegarder votre fichier. Lorsque cette commande est utilisée, vous serez invité à modifier ou vérifier le nom du fichier souhaité et après avoir appuyé sur Entrée, elle enregistrera votre fichier.
CTRL + X	Quitte l'éditeur de texte Nano. Dans le cas où vous avez apporté des modifications au fichier, elle vous invite à sauvegarder.

Comment ouvrir un fichier existant à l'aide de Nano ?

Vous pouvez utiliser la même commande que celle ci-dessus pour ouvrir un fichier existant. Rappel :

```
nano nomfichier
```

Il suffit alors d'exécuter nano avec le chemin d'accès au fichier que vous souhaitez ouvrir. Pour pouvoir éditer le fichier, vous devez avoir les permissions, sinon il s'ouvrira comme un fichier en lecture seule (en supposant que vous avez les permissions de lecture).

Utilisation de l'éditeur de texte nano avec Python

Dans le but de faciliter la lecture de code, la coloration syntaxique est pratiquement indispensable. Les mots-clés des langages sont mis en valeur, les commentaires un peu moins, etc... **nano** reconnaît alors le code python comme tel et colore les mots clés du python.

Dès que nano ouvre un fichier se terminant par .py il va utiliser cette coloration syntaxique.

Mais il y a aussi avec nano l'indentation automatique, et bien d'autres options qui peuvent être définies dans le fichier **~/.nanorc**

□ Ouvrez ce fichier de configuration de nano avec nano en tapant :

```
nano ~/.nanorc
```

Puis ajouter au fichier le texte suivant :

```
SET AUTOINDENT
SET TABSTOSPACES
SET TABSIZE 4
```

Signification :

SET AUTOINDENT: Indentation des nouvelles lignes en fonction de l'indentation de la ligne précédente. Cette fonction est également utile lors de l'édition du code source.

SET TABSTOSPACES: Convertir les tabulations dactylographiées en espaces.

SET TABSIZE 4: Définir la largeur d'une tabulations

□ Sauvegardez et quittez nano.

Pour tous les exercices suivants, créez un dossier TP_Python puis créez dedans un fichier par exercice avec nano et écrivez votre code.

Pour lancer l'exécution de votre code, il faut taper sous bash :

```
python3 nomdemonexercice.py
```

Affectations multiples

2.1 Décrivez le plus clairement et le plus complètement possible ce qui se passe à chacune des trois lignes de l'exemple ci-dessous :

```
largeur = 20
hauteur = 5 * 9.3
largeur * hauteur
```

2.2 Assignez les valeurs respectives 3, 5, 7 à trois variables a, b, c.
Effectuez l'opération a-b//c et affichez le résultat. Interprétez le résultat obtenu.

2.3 Testez le script composé des lignes d'instructions suivantes. Décrivez ce qui se passe :

```
r , pi = 12, 3.14159
s = pi * r**2
print(s)
print(type(r), type(pi), type(s))
```

Quelle est, à votre avis, l'utilité de la fonction type() ?

Exécution conditionnelle

3.1 Ecrivez un script qui, à partir de deux entiers a et b, affiche si a est supérieur ou non à b.

3.2 Ecrivez un programme qui affiche si une variable est positive, négative ou nulle.

3.3 Ecrivez un script qui teste si un entier est pair ou impair. Dans le premier cas, le programme affichera « a est pair » puis, sur la ligne suivante la justification mathématique associée. Sinon le programme affichera « a est impair »

Réaffectation

4.1 Supposons qu'un script comprenne l'instruction suivante :

```
a, b, c, d = 3, 4, 5, 7
```

Nous voulons maintenant échanger les valeurs des variables a et c (actuellement, après l'instruction, a contient la valeur 3, et c la valeur 5 ; nous voudrions que ce soit l'inverse). Écrivez les lignes d'instructions nécessaires pour obtenir ce résultat.

Commentaires

A partir de maintenant chacun de vos scripts contiendra au moins un commentaire de votre choix.

Répétitions en boucle – L'instruction while

4.2 Écrivez un programme qui affiche les 20 premiers termes de la table de multiplication par 7.

4.3 Écrivez un programme qui affiche une table de conversion de sommes d'argent exprimées en euros, en dollars canadiens. La progression des sommes de la table sera « géométrique », comme dans l'exemple ci-dessous :

```
1 euro(s) = 1.65 dollar(s)
2 euro(s) = 3.30 dollar(s)
4 euro(s) = 6.60 dollar(s)
8 euro(s) = 13.20 dollar(s)
```

etc. (S'arrêter à 16384 euros.)

4.4 Écrivez un programme qui affiche une suite de 12 nombres dont chaque terme soit égal au triple du terme précédent.

4.7 Écrivez un programme qui affiche les 20 premiers termes de la table de multiplication par 7, en signalant au passage (à l'aide d'une astérisque) ceux qui sont des multiples de 3. Exemple :

```
7 14 21 * 28 35 42 * 49 ...
```

4.8 Écrivez un programme qui calcule les 50 premiers termes de la table de multiplication par 13, mais n'affiche que ceux qui sont des multiples de 7.

4.9 Écrivez un programme qui affiche la suite de symboles suivante :

```
*
**
***
****
*****
*****
*****
```

Le type float

5.3 Écrivez un programme qui convertisse en degrés Celsius une température exprimée au départ en degrés Fahrenheit, ou l'inverse.

La formule de conversion est : $T_F = T_C \times 1,8 + 32$

5.4 Écrivez un programme qui calcule les intérêts accumulés chaque année pendant 20 ans, par capitalisation d'une somme de 100 euros placée en banque au taux fixe de 4,3 %

5.5 Une légende de l'Inde ancienne raconte que le jeu d'échecs a été inventé par un vieux sage, que son roi voulut remercier en lui affirmant qu'il lui accorderait n'importe quel cadeau en récompense. Le vieux sage demanda qu'on lui fournisse simplement un peu de riz pour ses vieux jours, et plus précisément un nombre de grains de riz suffisant pour que l'on puisse en déposer 1 seul sur la première case du jeu qu'il venait d'inventer, deux sur la suivante, quatre sur la troisième, et ainsi de suite jusqu'à la 64e case.

Écrivez un programme Python qui affiche le nombre de grains à déposer sur chacune des 64 cases du jeu. Calculez ce nombre de deux manières :

- le nombre exact de grains (nombre entier) ;
- le nombre de grains en notation scientifique (nombre réel).

Les données alphanumériques

5.6 Écrivez un script qui détermine si une chaîne contient ou non le caractère « e ».

5.7 Écrivez un script qui compte le nombre d'occurrences du caractère « e » dans une chaîne.

5.8 Écrivez un script qui recopie une chaîne (dans une nouvelle variable), en insérant des astérisques entre les caractères. Ainsi par exemple, « gaston » devra devenir « g*a*s*t*o*n »

5.9 Écrivez un script qui recopie une chaîne (dans une nouvelle variable) en l'inversant. Ainsi par exemple, « zorglub » deviendra « bulgroz ».

5.10 En partant de l'exercice précédent, écrivez un script qui détermine si une chaîne de caractères donnée est un palindrome (c'est-à-dire une chaîne qui peut se lire indifféremment dans les deux sens), comme par exemple « radar » ou « s.o.s ».

Les listes (première approche)

5.11 Soient les listes suivantes :

```
t1 = [31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31]
```

```
t2 = ['Janvier', 'Février', 'Mars', 'Avril', 'Mai', 'Juin', 'Juillet', 'Août', 'Septembre', 'Octobre', 'Novembre', 'Décembre']
```

Écrivez un petit programme qui crée une nouvelle liste t3. Celle-ci devra contenir tous les éléments des deux listes en les alternant, de telle manière que chaque nom de mois soit suivi du nombre de jours correspondant :

```
['Janvier',31,'Février',28,'Mars',31, etc...].
```

5.12 Écrivez un programme qui affiche « proprement » tous les éléments d'une liste. Si on l'appliquait par exemple à la liste t2 de l'exercice ci-dessus, on devrait obtenir : Janvier Février Mars Avril Mai Juin Juillet Août Septembre Octobre Novembre Décembre

5.13 Écrivez un programme qui recherche le plus grand élément présent dans une liste donnée. Par exemple, si on l'appliquait à la liste [32, 5, 12, 8, 3, 75, 2, 15], ce programme devrait afficher :

```
le plus grand élément de cette liste a la valeur 75.
```

5.14 Écrivez un programme qui analyse un par un tous les éléments d'une liste de nombres (par exemple celle de l'exercice précédent) pour générer deux nouvelles listes. L'une contiendra seulement les nombres pairs de la liste initiale, et l'autre les nombres impairs. Par exemple, si la liste initiale est celle de l'exercice précédent, le programme devra construire une liste pairs qui contiendra [32, 12, 8, 2], et une liste impairs qui contiendra [5, 3, 75, 15].

Astuce : pensez à utiliser l'opérateur modulo (%) déjà cité précédemment.

5.15 Écrivez un programme qui analyse un par un tous les éléments d'une liste de mots (par exemple : ['Jean', 'Maximilien', 'Brigitte', 'Sonia', 'Jean-Pierre', 'Sandra']) pour générer deux nouvelles listes. L'une contiendra les mots comportant moins de 6 caractères, l'autre les mots comportant 6 caractères ou davantage.

Fonctions prédéfinies

Dans tous ces exercices, utilisez la fonction input() pour l'entrée des données.

6.1 Écrivez un programme qui convertisse en mètres par seconde et en km/h une vitesse fournie par l'utilisateur en miles/heure. (Rappel : 1 mile = 1609 mètres)

6.2 Écrivez un programme qui calcule le périmètre et l'aire d'un triangle quelconque dont l'utilisateur fournit les 3 côtés.

(Rappel : l'aire d'un triangle quelconque se calcule à l'aide de la formule :

$S = \sqrt{d*(d-a)*(d-b)-(d-c)}$ dans laquelle d désigne la longueur du demi-périmètre, et a, b, c celles des trois côtés.)

6.4 Écrivez un programme qui permette d'encoder des valeurs dans une liste. Ce programme devrait fonctionner en boucle, l'utilisateur étant invité à entrer sans cesse de nouvelles valeurs, jusqu'à ce qu'il décide de terminer en frappant <Enter> en guise d'entrée. Le programme se terminerait alors par l'affichage de la liste. Exemple de fonctionnement :

```

Veillez entrer une valeur : 25
Veillez entrer une valeur : 18 Veuillez
entrer une valeur : 6284
Veillez entrer une valeur :
[25, 18, 6284]
```

Révisions

6.5 Que fait le programme ci-dessous, dans les quatre cas où l'on aurait défini au préalable que la variable a vaut 1, 2, 3 ou 15 ?

```

if a !=2:
    print('perdu') elif
a ==3:
    print('un instant, s.v.p.') else
: print('gagné')
```

6.6 Que font ces programmes ?

a)	<pre> a = 5 b = 2 if ((a==5) & (b<2)): print('"&" signifie "et"; on peut aussi utiliser\ le mot "and"')</pre>
b)	<pre> a, b = 2, 4 if ((a==4) or (b!=4)): print('gagné') elif ((a==4) or (b==4)): print('presque gagné')</pre>
c)	<pre> a = 1 if (not a) : print('gagné') elif (a) : print('perdu')</pre>

6.7 Reprendre le programme c) avec $a = 0$ au lieu de $a = 1$. Que se passe-t-il ? Conclure !

6.8 Écrire un programme qui, étant données deux bornes entières a et b, additionne les nombres multiples de 3 et de 5 compris entre ces bornes. Prendre par exemple a = 0, b = 32 ; le résultat devrait être alors 0 + 15 + 30 = 45.

Modifier légèrement ce programme pour qu'il additionne les nombres multiples de 3 ou de 5 compris entre les bornes a et b. Avec les bornes 0 et 32, le résultat devrait donc être : 0 + 3 + 5 + 6 + 9 + 10 + 12 + 15 + 18 + 20 + 21 + 24 + 25 + 27 + 30 = 225.

6.14 Soit la liste suivante : ['Jean-Michel', 'Marc', 'Vanessa', 'Anne', 'Maximilien', 'Alexandre-Benoît', 'Louise'] Écrivez un script qui affiche chacun de ces noms avec le nombre de caractères correspondant.

6.15 Écrire une boucle de programme qui demande à l'utilisateur d'entrer des notes d'élèves. La boucle se terminera seulement si l'utilisateur entre une valeur négative. Avec les notes ainsi entrées, construire progressivement une liste. Après chaque entrée d'une nouvelle note (et donc à chaque itération de la boucle), afficher le nombre de notes entrées, la note la plus élevée, la note la plus basse, la moyenne de toutes les notes.

Fonctions définies par l'utilisateur

7.2 Définissez une fonction `ligneCar(n, ca)` qui renvoie une chaîne de n caractères ca.

7.5 Définissez une fonction `maximum(n1, n2, n3)` qui renvoie le plus grand de 3 nombres n1, n2, n3 fournis en arguments. Par exemple, l'exécution de l'instruction :

```
print(maximum(2,5,4)) #donnera 5
```

7.9 Définissez une fonction `compteCar(ca, ch)` qui renvoie le nombre de fois que l'on rencontre le caractère ca dans la chaîne de caractères ch. Par exemple, l'exécution de l'instruction :

```
print(compteCar('e', 'Cette phrase est un exemple')) # affichera 7
```

7.10 Définissez une fonction `indexMax(liste)` qui renvoie l'index de l'élément ayant la valeur la plus élevée dans la liste transmise en argument. Exemple d'utilisation :

```
serie = [5, 8, 2, 1, 9, 3, 6, 7]
print(indexMax(serie)) #affichera 4
```

7.11 Définissez une fonction `nomMois(n)` qui renvoie le nom du énième mois de l'année. Par exemple, l'exécution de l'instruction :

```
print(nomMois(4)) #donnera Avril.
```

7.12 Définissez une fonction `inverse(ch)` qui permette d'inverser les l'ordre des caractères d'une chaîne quelconque. La chaîne inversée sera renvoyée au programme appelant.

7.13 Définissez une fonction `compteMots(ph)` qui renvoie le nombre de mots contenus dans la phrase ph. On considère comme mots les ensembles de caractères inclus entre des espaces.

Arguments avec étiquettes

7.14 Modifiez la fonction `volBoite(x1, x2, x3)` que vous avez définie dans un exercice précédent, de manière à ce qu'elle puisse être appelée avec trois, deux, un seul, ou même aucun argument. Utilisez pour ceux-ci des valeurs par défaut égales à 10. Par exemple :

```
print(volBoite())           #affichera 1000 print(volBoite(5.2))
#affichera 520.0 print(volBoite(5.2, 3)) #affichera 156.0
```

7.15 Modifiez la fonction `volBoite(x1,x2,x3)` ci-dessus de manière à ce qu'elle puisse être appelée avec un, deux, ou trois arguments. Si un seul est utilisé, la boîte est considérée comme cubique (l'argument étant l'arête de ce cube). Si deux sont utilisés, la boîte est considérée comme un prisme à base carrée (auquel cas le premier argument est le côté du carré, et le second la hauteur du prisme). Si trois arguments sont utilisés, la boîte est considérée comme un parallélépipède. Par exemple :

```
print(volBoite())           #affichera -1 (indication d'une erreur)
print(volBoite(5.2))       #affichera 140.608 print(volBoite(5.2,
3)) #affichera 81.12 print(volBoite(5.2, 3, 7.4))#affichera
115.44
```

7.16 Définissez une fonction `changeCar(ch, ca1, ca2, debut, fin)` qui remplace tous les caractères `ca1` par des caractères `ca2` dans la chaîne de caractères `ch`, à partir de l'indice `debut` et jusqu'à l'indice `fin`, ces deux derniers arguments pouvant être omis (et dans ce cas la chaîne est traitée d'une extrémité à l'autre). Exemples de la fonctionnalité attendue :

```
phrase = 'Ceci est une toute petite phrase.'
print(changeCar(phrase, ' ', '*'))
#affichera : Ceci*est*une*toute*petite*phrase.
print(changeCar(phrase, ' ', '*', 8, 12))
#affichera : Ceci est*une*toute petite phrase.
print(changeCar(phrase, ' ', '*', 12))
#affichera : Ceci est une*toute*petite*phrase.
print(changeCar(phrase, ' ', '*', fin = 12))
#affichera : Ceci*est*une*toute petite phrase.
```

7.17 Définissez une fonction `eleMax(liste, debut, fin)` qui renvoie l'élément ayant la plus grande valeur dans la liste transmise. Les deux arguments `debut` et `fin` indiqueront les indices entre lesquels doit s'exercer la recherche, et chacun d'eux pourra être omis (comme dans l'exercice précédent). Exemples de la fonctionnalité attendue :

```
serie = [9, 3, 6, 1, 7, 5, 4, 8, 2]
print(eleMax(serie))           #affichera 9
print(eleMax(serie, 2, 5))     #affichera 7
print(eleMax(serie, 2))        #affichera 8
print(eleMax(serie, fin =3, debut =1)) #affichera 6
```

Chaines et while

10.2 Découpez une grande chaîne en fragments de 5 caractères chacun. Rassemblez ces morceaux dans l'ordre inverse. La chaîne doit pouvoir contenir des caractères accentués.

10.3 Tâchez d'écrire une petite fonction `trouve()` qui fera exactement le contraire de ce que fait l'opérateur d'indexage (c'est-à-dire les crochets `[]`). Au lieu de partir d'un index donné pour retrouver le caractère correspondant, cette fonction devra retrouver l'index correspondant à un caractère donné.

En d'autres termes, il s'agit d'écrire une fonction qui attend deux arguments : le nom de la chaîne à traiter et le caractère à trouver. La fonction doit fournir en retour l'index du premier caractère de ce type dans la chaîne. Ainsi par exemple, l'instruction :

```
print(trouve("Juliette & Roméo", "&")) #devra afficher : 9
```

Attention : il faut penser à tous les cas possibles. Il faut notamment veiller à ce que la fonction renvoie une valeur particulière (par exemple la valeur -1) si le caractère recherché n'existe pas dans la chaîne traitée. Les caractères accentués doivent être acceptés.

10.4 Améliorez la fonction de l'exercice précédent en lui ajoutant un troisième paramètre : l'index à partir duquel la recherche doit s'effectuer dans la chaîne. Ainsi par exemple, l'instruction :

```
print(trouve ("César & Cléopâtre", "r", 5))  
#devra afficher : 15 (et non 4 !).
```

10.5 Écrivez une fonction `compteCar()` qui compte le nombre d'occurrences d'un caractère donné dans une chaîne. Ainsi :

```
print(compteCar("ananas au jus","a")) #devra afficher : 4  
print(compteCar("Gédéon est déjà là","é")) #devra afficher : 3
```

L'instruction `for ... in ...`

10.6 Dans un conte américain, huit petits canetons s'appellent respectivement : Jack, Kack, Lack, Mack, Nack, Oack, Pack et Qack. Écrivez un petit script qui génère tous ces noms à partir des deux chaînes suivantes :

```
prefixes = 'JKLMNOP' et suffixe = 'ack'
```

Si vous utilisez une instruction `for ... in ...`, votre script ne devrait comporter que deux lignes.

10.7 Dans un script, écrivez une fonction qui recherche le nombre de mots contenus dans une phrase donnée.

10.9 Écrivez une fonction `estUnChiffre()` qui renvoie vrai, si l'argument transmis est un chiffre, et faux sinon. Testez ainsi tous les caractères d'une chaîne en la parcourant à l'aide d'une boucle `for`.

10.10 Écrivez une fonction `estUneMaj()` qui renvoie vrai si l'argument transmis est une majuscule. Tâchez de tenir compte des majuscules accentuées !

10.11 Écrivez une fonction `chaineListe()` qui convertisse une phrase en une liste de mots.

10.12 Utilisez les fonctions définies dans les exercices précédents pour écrire un script qui puisse extraire d'un texte tous les mots qui commencent par une majuscule.

10.13 Utilisez les fonctions définies dans les exercices précédents pour écrire une fonction qui renvoie le nombre de caractères majuscules contenus dans une phrase donnée en argument.